



CoMIRATM
SOLUTIONS

RTL Assignment

NOTE: Please do the following to solve the assignment:

- a. Go through Chapters 3, 4 and 5 of the provided Ebook.
- b. Make and state necessary assumptions where required.
- c. Draw Block Diagrams and timing diagrams on paper where required.
- d. Write the RTL code. Don't forget to follow the coding guidelines.
- e. Write the testbench with automatic checkers and simulate the design on Questa/Modelsim.
- f. Use a 100 MHz clock where required.
- g. Your Verilog code should be synthesizable. Check that on Quartus Prime.
- h. Follow the given directory structure.



This is an incomplete sample of what your training repo should look like

- i. Keep your updated code on your git repository.
- j. Keep your code parameterized where signal width is not mentioned.
- k. You need to make a report in pdf format for every section. Your report should briefly explain your work. No need to make elaborate documents, a brief explanation with waveforms would be fine.

Section 1: Combinational Circuits

Estimated Completion Time = 4 days

1. Design a circuit that finds the Odd parity of a 16-bit number.
2. Implement a 4x1 multiplexer using case and if...else statements.
3. Implement the following combinational circuits as individual modules and connect them back to back in the test bench to get the same output as input.
 - a. Write a binary-to-gray converter.
 - b. Write a gray-to-binary converter.
4. Design a single bit full adder circuit. Instantiate the full adder in a wrapper using ***generate*** loop to create an n-bit full adder
5. Write a Verilog module which multiplies two 8-bit integers using Booth Multiplication Algorithm.
6. Design an 8-bit barrel shifter without using a shift operator

Section 2: Sequential Circuits

Estimated Completion Time = 10 days

1. Implement the following as individual modules. Instantiate all the modules in a single tb and simulate the designs.
 - a. Write a Verilog code to implement a delay pipeline for 32-bit data for N Delay cycles.
 - b. Write a Verilog code to detect the positive edge of a 1-bit signal **sig**. It should generate a one-shot signal after detecting a positive edge.
 - c. Write a Verilog code to detect the Negedge of a 1-bit signal **sig**. It should generate a one-shot signal after detecting a negedge.
 - d. Write a Verilog code to detect any change in a 32-bit wide signal. It should generate a one-shot signal after detecting a change.
 - e. Implement a circuit which gives out one pulse after every 100 ns time.
2. Implement 16-bit Parallel-In-Serial-Out circuit and 16-bit Parallel-Out-Serial-In circuit as separate Verilog modules. Connect these modules in the following configurations in a test-bench.
 - 2-1- Testbench -> PISO -> SIPO -> Testbench
 - 2-2 Testbench -> SIPO -> PISO -> Testbench
3. Write a Single Clock FIFO buffer 32wx128d (32-wide by 128 deep). The FIFO should have one cycle read latency. It should also indicate if it is full or empty. Implement System Verilog Queues in the testbench for verification. Also, try to keep the DEPTH and WIDTH of the FIFO parameterized.
4. Implement a module that can calculate the frequency of the input signal. The output should display the whole part of the frequency.

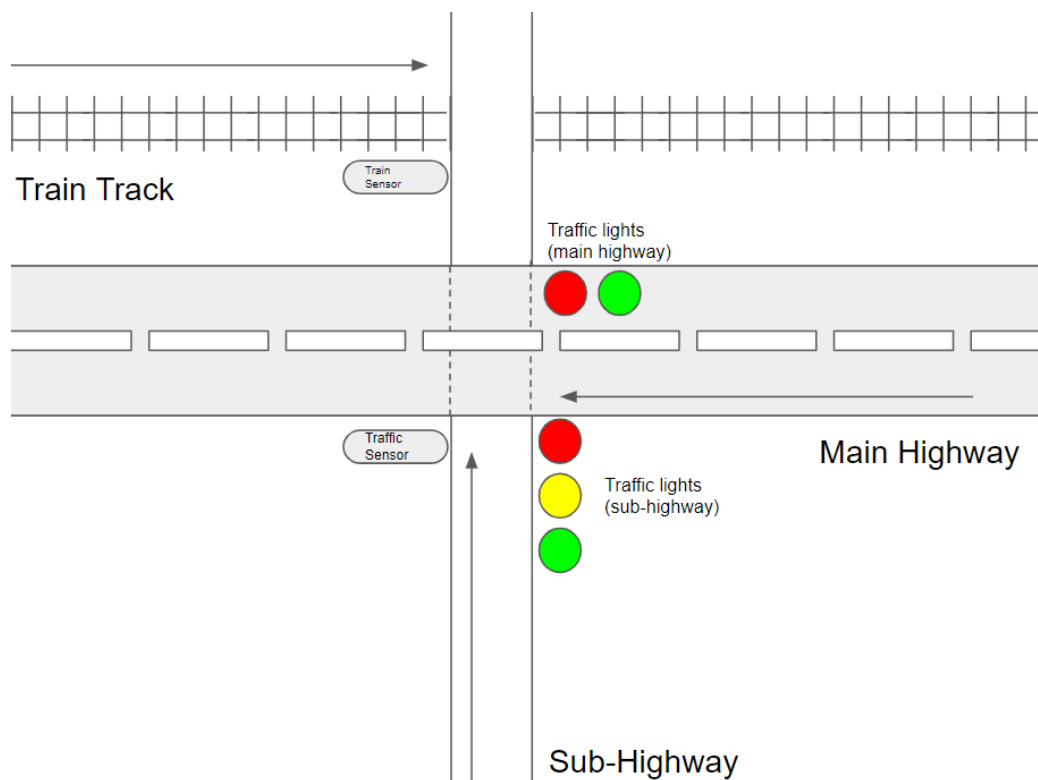
Note: The input signal's frequency can be from 1MHz to 50MHz.
5. Implement a random number generator using LFSR. It should have the following specifications
 - a. Its width should be 8-bit
 - b. You should be able to load a seed value to it
 - c. While in operation state, it should give a random number at every clock cycle

Generate 10,000 numbers with it and plot its histogram/distribution plot using a python script to visualize the degree of its randomness.

Section 3: FSM

Estimated Completion Time = 4 days

1. Design a traffic controller system for the following diagram



The following specifications must be considered during implementation

- Since cars are almost always present on main highway, its light should remain **green** by default
- Occasionally cars would come from the sub-highway, a sensor is present at the sub-highway that detects traffic, when traffic comes on the sub-highway, its traffic light should go from red to yellow and then green, meanwhile the main highway should turn red.
- Since there is very little traffic on sub-highway the traffic light should remain green to let all the traffic pass. After all the traffic has passed, the main highway signal should turn green.

- There is a train track parallel to the main highway, which crosses the sub-highway. There is a sensor that detects when the train is passing the tracks. When the train is passing, there is no need to check for traffic on sub-highway.
- You only need to check the traffic on sub-highway once the train has crossed.
- Every Light should remain on a color for 20 cycles by default. (This number of cycles should be parameterized)

2. Design a Finite State Machine to detect if the input stream contains a number divisible by 5.

Input: 0010101010

Output: 110011001